```
`default_nettype none
//////////////////////////////////////////////////////////////////////////////////
// Company: MIT
// Engineer: AHMAD TAKA
//
// Create Date: 10/10/2021 02:26:00 PM
// Design Name: CAR ALARM
// Module Name:
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments: This Lab was Actually Fun
//
//////////////////////////////////////////////////////////////////////////////////
module top_level(input wire clk_100mhz,
                                        input wire [15:0] sw,
                                        input wire btnc, btnu, btnl, btnd, btnr,
                                        output logic [1:0] led,
                                        output logic ca,cb,cc,cd,ce,cf,cg,
                                        output logic [7:0] an,
                                        output logic ja,
                                        output logic[3:0] vga_r,
                    output logic[3:0] vga_b,
                    output logic[3:0] vga_g,
                    output logic vga_hs,
                    output logic vga_vs
                    );
        parameter ONE_HZ_PERIOD = 25_000_000;
        parameter DEBOUNCE_COUNT = 1000000;
        parameter LOW_FREQ = 28409; //corresponds to 440hz
        parameter HIGH_FREQ = 14205; //corresponds to 880hz
    ////////////////////////////////////////////////////////////////
    // LOGICS
    ////////////////////////////////////////////////////////////////
    logic clk_25mhz;
    logic reset;
    assign reset = sw[15];

    //Buttons
    logic reprogram;
    logic h_switch;
    logic br_switch;
    logic dr_switch;
    logic p_switch;

    //Switches
    logic ig_switch;
    assign ig_switch = sw[14];
    logic [1:0] time_param;
    assign time_param = sw[5:4];
    logic [3:0] time_value;
    assign time_value = sw[3:0];

    //Multiplexer
    logic status_light;

    //Audio
    logic audio_signal; //used for audio generation (you control with siren generator)
    logic alarm;
```

```
    // FSM
    logic start_timer, status;
    logic [1:0] interval;
    logic [2:0] state;

    //Fuel Pump
     logic fuel_pump_led;

    //Timings
    logic [3:0] time_value_output, count;
    logic expired_pulse, counting, one_hz;

    //7 Seg
    logic [6:0] cat;
    logic [31:0] data;

    //Clock IN
     clk_25Mhz new_clk(.clk_in1(clk_100mhz),
        .clk_out1(clk_25mhz));

    //debounce instances:
        debounce #(.DEBOUNCE_COUNT(DEBOUNCE_COUNT)) db_reprogram  (.reset_in(reset), .
clock_in(clk_25mhz), .noisy_in(btnc),
            .clean_out(reprogram));
        debounce #(.DEBOUNCE_COUNT(DEBOUNCE_COUNT)) db_h_switch   (.reset_in(reset), .
clock_in(clk_25mhz), .noisy_in(btnu),
            .clean_out(h_switch));
        debounce #(.DEBOUNCE_COUNT(DEBOUNCE_COUNT)) db_br_switch  (.reset_in(reset), .
clock_in(clk_25mhz), .noisy_in(btnd),
            .clean_out(br_switch));
        debounce #(.DEBOUNCE_COUNT(DEBOUNCE_COUNT)) db_dr_switch  (.reset_in(reset), .
clock_in(clk_25mhz), .noisy_in(btnl),
            .clean_out(dr_switch));
    debounce #(.DEBOUNCE_COUNT(DEBOUNCE_COUNT)) db_p_switch   (.reset_in (reset), .clo
ck_in(clk_25mhz), .noisy_in(btnr),
        .clean_out(p_switch));

    //Time Parameter:
    time_parameter time1 (.clock_in(clk_25mhz),.reset_in(reset),.reprogram_in(reprogra
m), .interval_in(interval), .time_parameter_selector_in(time_param), .time_value_in(ti
me_value),
         .time_value_out(time_value_output));

    //Timer:
        timer #(.ONE_HZ_PERIOD(ONE_HZ_PERIOD)) time2 (.clock_in(clk_25mhz), .reset_in(
reset), .start_timer_in(start_timer),.value_in(time_value_output),
            .counting_out(counting),.expired_pulse_out(expired_pulse), .one_hz_out(one
_hz), .count_out(count));

    //Fuel Pump FSM:
    fuel_pump_logic fuel1(.clock_in(clk_25mhz), .reset_in(reset), .ignition_in(ig_swit
ch), .hidden_sw_in(h_switch), .brake_in(br_switch),
        .fuel_pump_led_out(fuel_pump_led));

    //Anti Theft FSM:
    anti_theft_fsm main_fsm (.clock_in(clk_25mhz),.reset_in(reset),.reprogram_in(repro
gram),.ignition_in(ig_switch),.driver_in(dr_switch),.passenger_in(p_switch),.time_expi
red_in(expired_pulse),
        .interval_out(interval),.state_out(state),.start_timer_out(start_timer),.alarm
_out(alarm),.status_out(status));

   //Audio Generator:
    siren_generator #(.HIGH_FREQ(HIGH_FREQ), .LOW_FREQ(LOW_FREQ), .ONE_HZ(ONE_HZ_PERIO
D)) wave_gen(.clock_in(clk_25mhz), .reset_in(reset), .alarm_in(alarm), .one_hz_in(one_
```

```
hz),
        .wave_out(audio_signal));

        //Multiplexer
        multiplexer #(.LED_INTERVAL(2)) multi(.clock_in(clk_25mhz), .reset_in(reset),
.led_status_in(status), .one_hz_in(one_hz),.state_in(state),
           .led_out(status_light));

        //Seven Segment Display
        seven_seg_controller seven_seg(.clk_in(clk_25mhz), .rst_in(reset), .val_in(dat
a),
           .cat_out(cat), .an_out(an));

        //Display Driver
        display_driver display(.clock_in(clk_25mhz), .reset_in(reset), .state_in(state
),
           .vga_r_out(vga_r), .vga_b_out(vga_b), .vga_g_out(vga_g), .vga_hs_out(vga_hs
), .vga_vs_out(vga_vs));

        assign data = {{1'b0, state},12'h0,count, 4'h0,{2'b0, time_param}, time_value}
;
        always_comb begin
           ca = cat[0];
           cb = cat[1];
           cc = cat[2];
           cd = cat[3];
           ce = cat[4];
           cf = cat[5];
           cg = cat[6];
           ja = audio_signal;
           led[0] = status_light;
           led[1] = fuel_pump_led;
        end
endmodule
//////////////////////////////////////////////////////////////////////////////////
//
// Pushbutton Debounce Module (LAB4 STARTER CODE)
//
//////////////////////////////////////////////////////////////////////////////////
module debounce (input wire reset_in, clock_in, noisy_in,
                 output logic clean_out);
   parameter DEBOUNCE_COUNT = 1000000;
   logic [19:0] count;
   logic new_input;

   always_ff @(posedge clock_in)
     if (reset_in) begin
        new_input <= noisy_in;
        clean_out <= noisy_in;
        count <= 0; end
     else if (noisy_in != new_input) begin new_input<=noisy_in; count <= 0; end
     else if (count == DEBOUNCE_COUNT) clean_out <= new_input;
     else count <= count+1;
endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// TIME PARAMETER MODULE
//
//////////////////////////////////////////////////////////////////////////////////
module time_parameter(
    input wire clock_in, reset_in, reprogram_in,
    input wire [1:0] interval_in, time_parameter_selector_in,
    input wire [3:0] time_value_in,
    output logic [3:0] time_value_out
```

```
    );
    logic [3:0] T_ARM_DELAY, T_DRIVER_DELAY, T_PASSENGER_DELAY, T_ALARM_ON;

    always_comb begin
        case(interval_in)
            //Combinational Logic for when signals come in from the fsm
            2'b00 : time_value_out = T_ARM_DELAY;
            2'b01 : time_value_out = T_DRIVER_DELAY;
            2'b10 : time_value_out = T_PASSENGER_DELAY;
            2'b11 : time_value_out = T_ALARM_ON;
            default : time_value_out = T_ARM_DELAY;
        endcase
    end
    always_ff @(posedge clock_in) begin
        //Logic For reprograming the time parameters
        if (reset_in) begin
            T_ARM_DELAY <= 4'b0110;
            T_DRIVER_DELAY <= 4'b1000;
            T_PASSENGER_DELAY <= 4'b1111;
            T_ALARM_ON <= 4'b1010;
        end else if (reprogram_in) begin
            case(time_parameter_selector_in)
                2'b00: T_ARM_DELAY <= time_value_in;
                2'b01: T_DRIVER_DELAY <= time_value_in;
                2'b10: T_PASSENGER_DELAY <= time_value_in;
                2'b11: T_ALARM_ON <= time_value_in;
                default: T_ARM_DELAY <= time_value_in;
            endcase
        end
    end
endmodule

////////////////////////////////////////////////////////////////////////////////
//
// TIMER MODULE
//
////////////////////////////////////////////////////////////////////////////////
module timer(   input wire clock_in, reset_in, start_timer_in,
                input wire [3:0] value_in,
                output logic counting_out,
                output logic expired_pulse_out,
                output logic one_hz_out,
                output logic [3:0] count_out);

        //parameter below for generating one_hz_enable signal
        parameter ONE_HZ_PERIOD = 25_000_000;
        logic [31:0] period_count;

        always_ff @(posedge clock_in) begin
            if (start_timer_in || reset_in) begin
                // Set All Outputs to defaults
                period_count <= 0;
            counting_out <= 1;
            count_out <= value_in;
            expired_pulse_out <= 0;
            one_hz_out <= 0;
        end else begin
            if (period_count == ONE_HZ_PERIOD - 1) begin
                //This is where one hz signal is enabled
                period_count <= 0;
                one_hz_out <= 1;
                expired_pulse_out <= 0;
                if (counting_out) begin
                    count_out <= count_out - 1; // if still counting decrement count
                end
```

```
            end else if (count_out == 0 && counting_out) begin
                //To reset the count values for the display we have to make sure we we
re still counting and count is currently at 0
                expired_pulse_out <= 1;
                counting_out <= 0;
                one_hz_out <= 0;
            end else begin
                //Default for icrementing the number of cycles to the ONE_HZ_PERIOD
                expired_pulse_out <= 0;
                one_hz_out <= 0;
                period_count <= period_count + 1;
            end
        end
    end
endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// FUEL PUMP FSM
//
//////////////////////////////////////////////////////////////////////////////////

module fuel_pump_logic(
     input wire clock_in, reset_in, ignition_in, hidden_sw_in, brake_in,
     output logic fuel_pump_led_out
    );
    enum {idle, waiting, led_on} state;
    always_ff @(posedge clock_in) begin
        if(reset_in) begin
            state <= idle;
        end else if (state == idle) begin
            fuel_pump_led_out <= 0;
            if(ignition_in) begin
                state <= waiting;
            end
        end else if (state == waiting) begin
            fuel_pump_led_out <= 0;
            if (hidden_sw_in && brake_in ) begin
                state <= led_on;
            end
        end else if (state == led_on) begin
            fuel_pump_led_out <= 1;
            if(!ignition_in) begin
                state <= idle;
            end
        end

    end

endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// MAIN FSM
//
//////////////////////////////////////////////////////////////////////////////////
module anti_theft_fsm(
    input wire clock_in, reset_in, reprogram_in, ignition_in, driver_in, passenger_in,
 time_expired_in,
    output logic [1:0] interval_out,
    output logic [2:0] state_out,
    output logic start_timer_out, alarm_out, status_out
    );
    enum {armed, disarmed, triggered, sound_alarm, sound_alarm_door_closed, disarmed_d
river_door_open, disarmed_driver_door_close} state;
```

```
    logic [1:0] led_count;
    always_ff @(posedge clock_in) begin
        if(reset_in || reprogram_in) begin
            //Default State
            state <= armed;
        end else if (state == armed) begin
            //Logic For Armed State
            alarm_out <= 0;
            status_out <= 1;
            state_out <= 3'b000;
            start_timer_out <= 0;
            if (ignition_in) begin
                state <= disarmed;
            end else if (driver_in) begin
                interval_out <= 2'b01;
                start_timer_out <= 1;
                state <= triggered;
            end else if (passenger_in) begin
                interval_out <= 2'b10;
                start_timer_out <= 1;
                state <= triggered;
            end
        end else if (state == triggered) begin
            //Logic for Triggered State
            alarm_out <= 0;
            status_out <= 1;
            state_out <= 3'b001;
            start_timer_out <= 0;
            if (ignition_in ) begin
                state <= disarmed;
            end else if (time_expired_in) begin

                state <= sound_alarm;
            end
        end else if (state == sound_alarm) begin
            //Logic For Sound Alarm State
            alarm_out <= 1;
            status_out <= 1;
            state_out <= 3'b010;
            start_timer_out <= 0;
            if (ignition_in) begin
                state <= disarmed;
            end else if (!passenger_in && !driver_in) begin
                start_timer_out <= 1;
                state <= sound_alarm_door_closed;
            end
        end else if(state == sound_alarm_door_closed) begin
           //Logic For when the Alarm is Sounding and the Doors are Closed
           alarm_out <= 1;
           status_out <= 1;
           state_out <= 3'b011;
           interval_out <= 2'b11;
           start_timer_out <= 0;
           if(time_expired_in) begin
                state <= armed;
           end
        end else if (state == disarmed ) begin
            //Logic For when in the disarmed state
            alarm_out <= 0;
            status_out <= 0;
            state_out <= 3'b100;
            start_timer_out <= 0;
            if (!ignition_in && driver_in) begin
                state <= disarmed_driver_door_open;
            end
```

```
        end else if (state == disarmed_driver_door_open) begin
            //Logic for when in the disarmed state and the driver door opens
            alarm_out <= 0;
            status_out <= 0;
            state_out <= 3'b101;
            start_timer_out <= 0;
            if (!driver_in) begin
               start_timer_out <= 1;
               state <= disarmed_driver_door_close;
            end
        end else if (state == disarmed_driver_door_close) begin
            //Logic for when in the disarmed state and the driver door closes
            alarm_out <= 0;
            status_out <= 0;
            state_out <= 3'b110;
            interval_out <= 2'b00;
            start_timer_out <= 0;
            if (time_expired_in) begin
                state <= armed;
            end
        end
    end

endmodule

////////////////////////////////////////////////////////////////////////////////
//
// SIREN GENERATOR
//
////////////////////////////////////////////////////////////////////////////////
module siren_generator(
    input wire clock_in, reset_in, alarm_in, one_hz_in,
    output logic wave_out
    );
    parameter HIGH_FREQ = 14205;
    parameter LOW_FREQ = 28409;
    parameter ONE_HZ = 25_000_000;

    logic [31:0] period_count_low;
    logic [31:0] period_count_high;
    logic state;
    always_ff @(posedge clock_in) begin
        if(reset_in) begin
            period_count_low <= 0;
            period_count_high <= 0;
            wave_out <= 0;
            state <= 0;
        end else if(alarm_in) begin
            if (one_hz_in) begin
                // if one hz is pulled high it changes states should happen every seco
nd
                 state <= ~state;

            end
            if (state) begin
                //High freqency counter
                if (period_count_high >= HIGH_FREQ) begin
                    wave_out <= ~wave_out;
                    period_count_high <= 0;
                end else begin
                    period_count_high <= period_count_high + 1;
                end
            end else begin
                //Low frequency counter
                if (period_count_low >= LOW_FREQ) begin
```

```
                    wave_out <= ~wave_out;
                    period_count_low <= 0;
                end else begin
                    period_count_low <= period_count_low + 1;
                end
            end
        end
    end
endmodule

////////////////////////////////////////////////////////////////////////////////
//
// MULTIPLEXER
//
////////////////////////////////////////////////////////////////////////////////

module multiplexer(
    input wire clock_in, reset_in, led_status_in, one_hz_in,
    input wire [2:0] state_in,
    output logic led_out
    );
    parameter LED_INTERVAL = 2;
    logic [1:0] period_count;
    always_ff @(posedge clock_in) begin
        if(reset_in) begin
            period_count <= 0;
            led_out <= 0;
        end else if(state_in == 3'b000 && led_status_in) begin
            // STATE is armed it will blink LED every LED_INTERVAL seconds
            if(one_hz_in) begin
                if (period_count == LED_INTERVAL - 1) begin
                   led_out <= 1;
                   period_count <= 0;
                end else begin
                   period_count <= period_count + 1;
                   led_out <= 0;
                end
            end
        end else if(led_status_in) begin
            //IF status is high but its not in armed the LED should be solid
            led_out <= 1;
        end else begin
            //else set to low
            led_out <= 0;
        end
    end
endmodule

////////////////////////////////////////////////////////////////////////////////
//
// 7 SEGMENT CONTROLLER FROM  LAB2
//
////////////////////////////////////////////////////////////////////////////////
module seven_seg_controller(input wire         clk_in,
                            input wire         rst_in,
                            input wire [31:0]  val_in,
                            output logic [6:0]   cat_out,
                            output logic [7:0]   an_out
    );

    logic [7:0]     segment_state;
    logic [31:0]    segment_counter;
    logic [3:0]     routed_vals;
    logic [6:0]     led_out;
```

```
    binary_to_seven_seg my_converter ( .bin_in(routed_vals), .hex_out(led_out));
    assign cat_out = ~led_out;
    assign an_out = ~segment_state;


    always_comb begin
        case(segment_state)
            8'b0000_0001:   routed_vals = val_in[3:0];
            8'b0000_0010:   routed_vals = val_in[7:4];
            8'b0000_0100:   routed_vals = val_in[11:8];
            8'b0000_1000:   routed_vals = val_in[15:12];
            8'b0001_0000:   routed_vals = val_in[19:16];
            8'b0010_0000:   routed_vals = val_in[23:20];
            8'b0100_0000:   routed_vals = val_in[27:24];
            8'b1000_0000:   routed_vals = val_in[31:28];
            default:        routed_vals = val_in[3:0];
        endcase
    end

    always_ff @(posedge clk_in)begin
        if (rst_in)begin
            segment_state <= 8'b0000_0001;
            segment_counter <= 32'b0;
        end else begin
            if (segment_counter == 32'd100_000)begin
                segment_counter <= 32'd0;
                segment_state <= {segment_state[6:0],segment_state[7]};
            end else begin
                segment_counter <= segment_counter +1;
            end
        end
    end

endmodule //seven_seg_controller


//feel free to either include binary_to_seven_seg module here or in its own file!
module binary_to_seven_seg (
        input wire [3:0]       bin_in,  //declaring input explicitely
        output logic [6:0]      hex_out);  //declaring output explicitely


        always @(bin_in) begin
            case (bin_in)
                4'b0000 : hex_out = 7'h3F;
                4'b0001 : hex_out = 7'h06;
                4'b0010 : hex_out = 7'h5B;
                4'b0011 : hex_out = 7'h4F;

                4'b0100 : hex_out = 7'h66;
                4'b0101 : hex_out = 7'h6D;
                4'b0110 : hex_out = 7'h7D;
                4'b0111 : hex_out = 7'h07;

                4'b1000 : hex_out = 7'h7F;
                4'b1001 : hex_out = 7'h67;
                4'b1010 : hex_out = 7'h77;
                4'b1011 : hex_out = 7'h7C;

                4'b1100 : hex_out = 7'h39;
                4'b1101 : hex_out = 7'h5E;
                4'b1110 : hex_out = 7'h79;
                4'b1111 : hex_out = 7'h71;
            endcase
        end
```

```
endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// DISPLAY DRIVER COPIED STUFF FROM PREVIOUS LABS AND PSET
//
//////////////////////////////////////////////////////////////////////////////////
module display_driver(
        input wire clock_in, reset_in,
        input wire [2:0] state_in,
        output logic[3:0] vga_r_out,
        output logic[3:0] vga_b_out,
        output logic[3:0] vga_g_out,
        output logic vga_hs_out,
        output logic vga_vs_out
        );
        logic [10:0] hcount;    // pixel on current line
        logic [9:0] vcount;     // line number
        logic hsync, vsync, blank; //control signals for vga
        logic [11:0] pixel;
        logic [11:0] rgb;
        logic phsync,pvsync,pblank;
        logic b,hs,vs;
        parameter WIDTH = 640;
        parameter HEIGHT = 480;

        xvga xvga1(.vclock_in(clock_in),.hcount_out(hcount),.vcount_out(vcount),
              .hsync_out(hsync),.vsync_out(vsync),.blank_out(blank));

        display_state #(.S_WIDTH(WIDTH), .S_HEIGHT(HEIGHT)) ds(.vclock_in(clock_in),.r
eset_in(reset_in),
                .hcount_in(hcount),.vcount_in(vcount),
                .hsync_in(hsync),.vsync_in(vsync),.blank_in(blank),
                .state_in(state_in),
                .phsync_out(phsync),.pvsync_out(pvsync),.pblank_out(pblank),.pixel_out
(pixel));



        always_ff @(posedge clock_in) begin

             hs <= phsync;
             vs <= pvsync;
             b <= pblank;
             rgb <= pixel;
        end

        // the following lines are required for the Nexys4 VGA circuit - do not change
        assign vga_r_out = ~b ? rgb[11:8]: 0;
        assign vga_g_out = ~b ? rgb[7:4] : 0;
        assign vga_b_out = ~b ? rgb[3:0] : 0;

        assign vga_hs_out = ~hs;
        assign vga_vs_out = ~vs;

endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// state_display: the logic for the display itself!
//
//////////////////////////////////////////////////////////////////////////////////

module display_state (
   input wire vclock_in,        // 25MHz clock
```

```
   input wire reset_in,          // 1 to initialize module
   input wire [10:0] hcount_in, // horizontal index of current pixel (0..1023)
   input wire [9:0]  vcount_in, // vertical index of current pixel (0..767)
   input wire hsync_in,          // XVGA horizontal sync signal (active low)
   input wire vsync_in,          // XVGA vertical sync signal (active low)
   input wire blank_in,          // XVGA blanking (1 means output black pixel)
   input wire [2:0] state_in,

   output logic phsync_out,       // pong game's horizontal sync
   output logic pvsync_out,       // pong game's vertical sync
   output logic pblank_out,       // pong game's blanking
   output logic [11:0] pixel_out  // pong game's pixel  // r=11:8, g=7:4, b=3:0
   );
   parameter S_WIDTH = 640;
   parameter S_HEIGHT = 480;
   parameter GLOBAL_RADIUS = 64;
   parameter FLASH_PERIOD = 25_000_000;

   logic [10:0] global_x;
   assign global_x = (S_WIDTH >> 2) + GLOBAL_RADIUS;
   logic [9:0] global_y;
   assign global_y = (S_HEIGHT >> 2) + GLOBAL_RADIUS;


   logic [11:0] disarm_pixel, waiting_pixel, alarm_pixel;
   // REPLACE ME! The code below just generates a color checkerboard
   // using 64 pixel by 64 pixel squares.
   circle #(.RADIUS(GLOBAL_RADIUS),.COLOR(12'h0F0)) disarm(.x(global_x),.y(global_y),.
hcount(hcount_in),.vcount(vcount_in),.pixel_out(disarm_pixel));
   circle #(.RADIUS(GLOBAL_RADIUS),.COLOR(12'h00F)) waiting(.x(global_x),.y(global_y),
.hcount(hcount_in),.vcount(vcount_in),.pixel_out(waiting_pixel));
   circle #(.RADIUS(GLOBAL_RADIUS),.COLOR(12'hF00)) alarm(.x(global_x),.y(global_y),.h
count(hcount_in),.vcount(vcount_in),.pixel_out(alarm_pixel));

   assign phsync_out = hsync_in;
   assign pvsync_out = vsync_in;
   assign pblank_out = blank_in;
   logic [31:0] period_count;
   logic alarm_on;
   always_comb begin
        case(state_in)
            3'b000: pixel_out = alarm_pixel;
            3'b100: pixel_out = disarm_pixel;
            3'b010: pixel_out = (alarm_on)? alarm_pixel : 12'b0;
            3'b011: pixel_out = (alarm_on)? alarm_pixel : 12'b0;
            default: pixel_out = waiting_pixel;
        endcase

    end
    always_ff @(posedge vclock_in) begin
        if(reset_in) begin
            period_count <= 0;
        end else begin
            if (period_count >= FLASH_PERIOD) begin
                period_count <= 0;
                alarm_on <= ~alarm_on;
            end else begin
                period_count <= period_count + 1;
            end
        end
    end


endmodule
//////////////////////////////////////////////////////////////////////////////
```

```
//
// CIRCle: the logic for the display itself!
//
//////////////////////////////////////////////////////////////////////////////
module circle (
        input wire [10:0] x,hcount,
        input wire [9:0] y,vcount,
        output logic [11:0] pixel_out);

    parameter  RADIUS = 64;           // default height: 100 pixels
    parameter  COLOR = 12'hFFF;
    parameter RASIUS_SQUARED = RADIUS*RADIUS;
    logic [10: 0] deltax; // can't be bigger than the screen
    logic [9: 0] deltay; // can't be bigger than the screen

    always_comb begin // generate round puck
            // compute x-xcenter and y-ycenter
            deltax = (hcount > (x+RADIUS)) ? (hcount-(x+RADIUS)) : ((x+RADIUS)-hcount)
;
            deltay = (vcount > (y+RADIUS)) ? (vcount-(y+RADIUS)) : ((y+RADIUS)-vcount)
;
            // check if distance is less than radius squared
            if(deltax*deltax + deltay*deltay <= RASIUS_SQUARED) begin
                  pixel_out = COLOR;
            end else begin
                  pixel_out = 0;
            end
    end
endmodule


//////////////////////////////////////////////////////////////////////////////
// Update: 8/8/2019 GH
// Create Date: 10/02/2015 02:05:19 AM
// Module Name: xvga
//
// xvga: Generate VGA display signals (1024 x 768 @ 60Hz)
//
//                              ---- HORIZONTAL -----     ------VERTICAL -----
//                              Active                    Active
//                    Freq      Video   FP  Sync   BP      Video   FP  Sync  BP
//   640x480, 60Hz    25.175    640     16    96   48       480    11   2    31
//   800x600, 60Hz    40.000    800     40   128   88       600     1   4    23
//   1024x768, 60Hz   65.000    1024    24   136  160       768     3   6    29
//   1280x1024, 60Hz  108.00    1280    48   112  248       768     1   3    38
//   1280x720p 60Hz   75.25     1280    72    80  216       720     3   5    30
//   1920x1080 60Hz   148.5     1920    88    44  148      1080     4   5    36
//
// change the clock frequency, front porches, sync's, and back porches to create
// other screen resolutions
//////////////////////////////////////////////////////////////////////////////

module xvga(input wire vclock_in,
            output logic [10:0] hcount_out,    // pixel number on current line
            output logic [9:0] vcount_out,     // line number
            output logic vsync_out, hsync_out,
            output logic blank_out);

       parameter DISPLAY_WIDTH  = 640;      // display width
       parameter DISPLAY_HEIGHT = 480;       // number of lines

       parameter  H_FP = 16;                 // horizontal front porch
       parameter  H_SYNC_PULSE = 96;        // horizontal sync
       parameter  H_BP = 48;                // horizontal back porch
```

```
       parameter  V_FP = 11;                  // vertical front porch
       parameter  V_SYNC_PULSE = 2;          // vertical sync
       parameter  V_BP = 31;                 // vertical back porch

       // horizontal: 1344 pixels total
       // display 1024 pixels per line
       logic hblank,vblank;
       logic hsyncon,hsyncoff,hreset,hblankon;
       assign hblankon = (hcount_out == (DISPLAY_WIDTH -1));
       assign hsyncon = (hcount_out == (DISPLAY_WIDTH + H_FP - 1));  //1047
       assign hsyncoff = (hcount_out == (DISPLAY_WIDTH + H_FP + H_SYNC_PULSE - 1));  /
/ 1183
       assign hreset = (hcount_out == (DISPLAY_WIDTH + H_FP + H_SYNC_PULSE + H_BP - 1)
);  //1343

       // vertical: 806 lines total
       // display 768 lines
       logic vsyncon,vsyncoff,vreset,vblankon;
       assign vblankon = hreset & (vcount_out == (DISPLAY_HEIGHT - 1));    // 767
       assign vsyncon = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP - 1));  // 771
       assign vsyncoff = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP + V_SYNC_PULSE
 - 1));  // 777
       assign vreset = hreset & (vcount_out == (DISPLAY_HEIGHT + V_FP + V_SYNC_PULSE +
 V_BP - 1)); // 805

       // sync and blanking
       logic next_hblank,next_vblank;
       assign next_hblank = hreset ? 0 : hblankon ? 1 : hblank;
       assign next_vblank = vreset ? 0 : vblankon ? 1 : vblank;
       always_ff @(posedge vclock_in) begin
          hcount_out <= hreset ? 0 : hcount_out + 1;
          hblank <= next_hblank;
          hsync_out <= hsyncon ? 0 : hsyncoff ? 1 : hsync_out;  // active low

          vcount_out <= hreset ? (vreset ? 0 : vcount_out + 1) : vcount_out;
          vblank <= next_vblank;
          vsync_out <= vsyncon ? 0 : vsyncoff ? 1 : vsync_out;  // active low

          blank_out <= next_vblank | (next_hblank & ~hreset);
       end
endmodule


`default_nettype wire
```